

Informe Final del Compilador de HULK

Diseño del frontend, análisis semántico, IR, backend LLVM, runtime y
extensión de lenguaje

Camilo Pérez Fleita
Guillermo Hugues Cardona
Mauricio Medina Hernández

Proyecto de Compilación / Lenguajes de Programación

21 de junio de 2026

Resumen

Este informe presenta el diseño y la implementación de un compilador para HULK construido como un sistema por fases: frontend léxico y sintáctico, AST, análisis semántico, HIR tipada, lowering a una representación intermedia propia, backend LLVM y runtime de soporte escrito en C. El énfasis del documento no está solamente en enumerar componentes del repositorio, sino en justificar las decisiones de diseño tomadas en cada frontera del compilador: por qué se usa un lexer generado, por qué se combina un parser LL(1) con un Pratt parser, por qué se introduce una HIR tipada antes del lowering, por qué se define una IR propia antes de emitir LLVM y por qué ciertas operaciones se delegan a un runtime en C. Desde la perspectiva de Lenguajes de Programación, la extensión propia principal es la incorporación de `match` como expresión con patrones, bindings, narrowing de tipos y chequeo conservador de exhaustividad. El objetivo del informe es presentar de forma formal el alcance técnico del compilador, las garantías que ofrece, los compromisos asumidos y las limitaciones que quedan como trabajo futuro.

Índice

1. Introducción	5
2. Alcance del compilador	5
2.1. Propósito general	5
2.2. Lenguaje soportado	6
2.3. Alcance técnico	7
2.4. Pipeline de alto nivel	7

3. Frontend	8
3.1. Decisión léxica: lexer generado	8
3.2. Decisión sintáctica: LL(1) para estructura general	9
3.3. Decisión expresiva: Pratt parser para expresiones	9
3.4. AST como frontera entre sintaxis y semántica	10
3.5. Separación de categorías de error	10
4. Análisis semántico e HIR	11
4.1. Registro de tipos y análisis en varias pasadas	11
4.2. Inferencia iterativa	11
4.3. HIR tipada como contrato interno	12
4.4. Chequeo de objetos, herencia y protocolos	12
4.5. Errores semánticos diferenciados	13
5. Representación intermedia y lowering	13
5.1. Motivación de una IR propia	13
5.2. Relación con BANNER	14
5.3. Forma general de la IR	14
5.4. Instrucciones principales	14
5.5. Lowering de objetos y métodos	15
5.6. Lowering de base	15
5.7. Lowering de vectores, rangos y ciclos	16
5.8. Lowering de lambdas y closures	16
6. Backend LLVM y runtime	16
6.1. Motivación de LLVM como backend	16
6.2. Emisión de LLVM IR	17
6.3. Papel del runtime en C	17
6.4. El runtime no es una máquina virtual	17
6.5. Driver y modos de inspección	18
7. Extensión propia de lenguaje: match	18
7.1. Problema de diseño	18

7.2. Alternativas consideradas	19
7.3. Sintaxis y semántica básica	19
7.4. Patrones soportados	20
7.5. Narrowing de tipos	20
7.6. Exhaustividad conservadora	21
7.7. Desugaring a construcciones existentes	21
7.8. Comparación con otros lenguajes	22
8. Mecanismos avanzados de HULK integrados	23
8.1. Protocolos estructurales	23
8.2. <code>interface</code> como alias de <code>protocol</code>	23
8.3. Varianza en firmas de protocolos	24
8.4. Vectores, arreglos e iterables	24
8.5. Lambdas, functors y closures	24
9. Validación y evidencia experimental	25
9.1. Estrategia de pruebas por capas	25
9.2. Validación de <code>match</code>	25
9.3. Validación del backend y runtime	26
9.4. Comandos de validación	26
10.Limitaciones y trabajo futuro	27
10.1. Gestión de memoria	27
10.2. Exhaustividad de <code>match</code>	27
10.3. Protocolos y conformidad accidental	27
11.Conclusiones	27
A. Comandos de uso recomendados	28
A.1. Construcción del compilador	28
A.2. Compilación de un programa HULK	28
A.3. Inspección por fases	29
B. Programa demostrativo	29

1. Introducción

Construir un compilador para HULK no consiste solamente en transformar texto de entrada en otro texto de salida. El objetivo central es preservar el significado de un programa a través de varias capas de representación. Un programa fuente con expresiones, funciones, objetos, herencia, protocolos, iterables, operaciones de tipo y una extensión como `match` debe convertirse gradualmente en estructuras internas verificables y, finalmente, en código de bajo nivel capaz de ejecutarse como binario nativo.

El sistema implementa una arquitectura por fases. Primero, el frontend reconoce tokens y estructura sintáctica; después, el AST elimina detalles de la gramática concreta; luego, la fase semántica resuelve nombres, scopes, tipos, atributos, métodos, `self`, `base` y builtins; más adelante, la HIR conserva la estructura de alto nivel pero ya tipada; el lowering transforma esa HIR en una representación intermedia propia; finalmente, el backend LLVM emite un módulo enlazable con una biblioteca C de soporte.

El compilador permite compilar programas HULK hasta ejecutables mediante LLVM y `clang`. También conserva modos de inspección para mostrar AST, chequeo semántico, HIR, IR y LLVM IR. Esta doble función es importante desde el punto de vista académico: el sistema no se comporta como una caja negra, sino como un conjunto de transformaciones observables. Cada representación intermedia permite explicar y validar una fase distinta del proceso de compilación.

Este informe no se organiza como una descripción cronológica del repositorio ni como una lista de archivos. Su estructura sigue una idea de diseño: en cada fase se explica qué problema debía resolverse, qué alternativas existían, qué decisión se adoptó y qué consecuencias tiene esa decisión para las fases posteriores. Por esa razón, la arquitectura en Rust se presenta solo como soporte físico de la implementación; el centro del análisis son las decisiones de compilación y de diseño de lenguaje.

Desde Lenguajes de Programación, el aporte propio principal es `match`. Esta construcción introduce una forma declarativa de clasificación de valores mediante patrones. Su relevancia no es únicamente sintáctica: modifica el AST, participa en resolución de scopes, introduce bindings locales, permite narrowing de tipos, requiere chequeo de exhaustividad y se integra al pipeline mediante desugaring hacia construcciones existentes. Otros mecanismos avanzados de HULK, como protocolos estructurales, vectores, iterables, lambdas y closures, se analizan como parte del lenguaje soportado y como contexto necesario para entender el alcance del compilador.

2. Alcance del compilador

2.1. Propósito general

El propósito del compilador es transformar programas escritos en HULK en código LLVM IR y, posteriormente, en un binario ejecutable enlazado con un runtime de soporte. Para lograrlo,

el sistema no realiza una traducción directa desde el código fuente hasta LLVM, sino una sucesión de representaciones con responsabilidades separadas: tokens, CST, AST, HIR tipada, IR propia y LLVM IR.

Esta separación permite que cada fase tenga un contrato claro. El frontend debe reconocer la estructura del programa; la semántica debe rechazar programas inválidos y producir información de tipos; la HIR debe fijar decisiones semánticas; la IR debe hacer explícito el control de flujo y las operaciones de bajo nivel; el backend debe emitir LLVM correcto; y el runtime debe proveer operaciones que no conviene repetir manualmente en LLVM textual.

2.2. Lenguaje soportado

El compilador cubre el núcleo expresivo de HULK y varios mecanismos avanzados. El lenguaje soportado incluye literales numéricos, booleanos y strings; operadores aritméticos, booleanos y de concatenación; bloques; variables con `let`; asignación destructiva; condicionales; ciclos `while` y `for`; funciones globales; llamadas recursivas; objetos; atributos; métodos; herencia simple; despacho dinámico; `self`; `base`; pruebas dinámicas de tipo con `is`; casts con `as`; protocolos; vectores; iterables; lambdas; closures; y expresiones `match`.

La expresión final del archivo funciona como punto de entrada del programa. Este rasgo obliga a tratar condicionales, ciclos, bloques, llamadas y `match` como expresiones con tipo y valor resultante, no solo como instrucciones. Por tanto, el análisis semántico debe calcular tipos para expresiones compuestas y el lowering debe preservar el valor producido por cada construcción.

Listing 1: Programa simple con funciones, objetos, herencia y match

```
function square(x: Number): Number => x * x;

type Shape() {
  area(): Number => 0;
}

type Circle(r: Number) inherits Shape {
  r: Number = r;
  area(): Number => 3 * square(self.r);
}

function describe(s: Shape): String =>
  match (s) {
    Circle as c => "circle area = " @ c.area(),
    _ => "unknown shape"
  };

let s: Shape = new Circle(4) in print(describe(s));
```

2.3. Alcance técnico

El compilador principal está implementado en Rust y el runtime de soporte está implementado en C. La estructura física del repositorio refleja las fases del compilador: existen crates separados para generación léxica, generación sintáctica, frontend, semántica, IR, lowering, backend LLVM y driver. Sin embargo, esta división no es el objetivo central del diseño, sino un medio para mantener límites claros entre responsabilidades.

Componente	Responsabilidad principal
<code>hulk-lexgen</code>	Generación y runtime de lexer a partir de especificaciones léxicas.
<code>hulk-parsegen</code>	Generación de parser, cálculo de estructuras gramaticales y runtime sintáctico.
<code>hulk-frontend</code>	API pública de parseo, construcción de CST y conversión a AST.
<code>hulk-sema</code>	Resolución de nombres, registro de tipos, inferencia, chequeo semántico y HIR tipada.
<code>hulk-ir</code>	Modelo de la representación intermedia propia.
<code>hulk-lower</code>	Traducción desde el programa semántico hacia IR.
<code>hulk-codegen-llvm</code>	Emisión textual de LLVM IR.
<code>hulk-driver</code>	Integración del pipeline, modos de depuración y compilación final.
<code>runtime/</code>	Biblioteca C de soporte para ejecución.

2.4. Pipeline de alto nivel

El pipeline completo puede resumirse de la siguiente manera:

Listing 2: Pipeline conceptual del compilador

```
archivo .hulk
-> lexer generado desde especificaciones
-> tokens
-> parser LL(1) + Pratt parser
-> CST
-> AST
-> análisis semántico
-> HIR tipada + TypeRegistry
-> lowering
-> Hulk IR (.TYPES, .DATA, .CODE)
-> LLVM IR
-> clang + runtime/hulk_runtime.c
-> ejecutable
```

El diseño se apoya en una idea clásica de compilación: no intentar resolver todos los problemas en una sola representación. La sintaxis concreta es útil para reconocer el programa, pero incómoda para razonar sobre tipos; el AST es cómodo para semántica, pero demasiado alto nivel para emitir LLVM; LLVM es adecuado para backend, pero demasiado bajo nivel para

representar directamente algunas decisiones del lenguaje. Por eso se introducen fronteras intermedias.

3. Frontend

El frontend transforma una secuencia de caracteres en una representación estructurada del programa. Esta fase debe resolver dos problemas diferentes. El primero es léxico: identificar palabras clave, operadores, delimitadores, identificadores, literales, comentarios y espacios. El segundo es sintáctico: reconocer cómo esos tokens forman declaraciones, expresiones, tipos, protocolos, bloques y patrones.

Una implementación monolítica que combine lexing, parsing y construcción de AST sería posible, pero tendría dos desventajas importantes. Primero, mezclaría responsabilidades: un error léxico, un conflicto gramatical y una decisión de AST aparecerían en el mismo código. Segundo, dificultaría el crecimiento del lenguaje, porque agregar una construcción como `match` obligaría a modificar demasiados lugares sin una frontera clara.

3.1. Decisión léxica: lexer generado

El sistema implementa un lexer generado a partir de especificaciones. Esta decisión es adecuada porque el conjunto léxico de HULK no se reduce a identificadores y números. El lenguaje contiene palabras clave como `function`, `define`, `type`, `inherits`, `new`, `self`, `base`, `let`, `if`, `while`, `for`, `protocol`, `interface`, `extends` y `match`; operadores como `@@`, `@`, `==`, `!=`, `<=`, `:=`, `=>` y `->`; delimitadores; literales numéricos; literales string; comentarios; y whitespace.

Un lexer manual daría control fino sobre cada caso, pero también haría que la definición del lenguaje quedara dispersa en código imperativo. Al usar un generador, la especificación léxica se vuelve más declarativa: las reglas que describen tokens se mantienen separadas del runtime que los consume. Esto facilita agregar palabras clave nuevas, ajustar operadores o cambiar reglas de literales sin reescribir el ciclo principal de reconocimiento.

Desde el punto de vista teórico, esta decisión conecta la implementación con la idea de autómatas finitos para reconocimiento léxico. El lexer generado puede entenderse como una automatización del paso desde expresiones regulares de tokens hacia una máquina que recorre el texto fuente. Desde el punto de vista de ingeniería, reduce errores repetitivos y mantiene una interfaz estable para el parser: una secuencia de tokens con posición de origen.

Frente a herramientas externas como Lex/Flex, el generador propio tiene una ventaja didáctica: permite mostrar dentro del proyecto cómo se representa y ejecuta la especificación léxica. Frente a un lexer completamente escrito a mano, tiene una ventaja de mantenibilidad: el vocabulario del lenguaje vive en una especificación más cercana a la gramática del lenguaje que al código de bajo nivel.

3.2. Decisión sintáctica: LL(1) para estructura general

Para la estructura superior del programa, el sistema usa una estrategia LL(1). Esta elección es razonable porque muchas partes de HULK tienen una forma sintáctica reconocible por el primer token: una declaración de función comienza con `function` o `define`; una declaración de tipo comienza con `type`; un protocolo comienza con `protocol` o `interface`; un bloque comienza con llave; y varias construcciones de control tienen palabras clave propias.

Un parser LL(1) tiene valor académico porque obliga a pensar la gramática en términos de predicción, FIRST/FOLLOW y ausencia de conflictos. Esto hace explícita una parte importante del diseño sintáctico. Cuando la gramática es adecuada para LL(1), el parser resultante es simple, determinista y fácil de depurar: en cada punto se decide qué producción aplicar mirando el no terminal actual y el token de entrada.

La alternativa de usar un parser LR o LALR externo habría permitido aceptar gramáticas más amplias, pero habría reducido el control didáctico sobre la generación del parser. La alternativa de escribir un parser recursivo manual completo también era viable, pero habría mezclado la definición de la gramática con detalles de implementación. En este contexto, LL(1) funciona bien como núcleo formal para la estructura general del lenguaje.

3.3. Decisión expresiva: Pratt parser para expresiones

Aunque LL(1) es adecuado para la estructura general, no es la mejor herramienta para toda la sintaxis de expresiones de HULK. Las expresiones tienen precedencia, asociatividad, operadores prefijos e infijos, llamadas, acceso por punto, indexación, casts con `as`, pruebas de tipo con `is`, lambdas y construcciones como `match`. Codificar todos esos niveles exclusivamente en una gramática LL(1) produciría muchas reglas intermedias y haría más costoso extender el lenguaje.

Por esa razón, el parser combina la gramática LL(1) con un Pratt parser para expresiones. Un Pratt parser organiza el análisis alrededor de precedencias y poderes de enlace. Esto permite manejar de forma natural operadores infijos como `+`, `*`, `==`, `@`; operadores prefijos como `!` o `-`; y operadores postfijos o encadenados como llamadas, indexación y acceso a miembros.

La combinación de LL(1) y Pratt parser es un compromiso equilibrado. El LL(1) conserva formalidad para declaraciones y bloques; el Pratt parser aporta flexibilidad para expresiones ricas. Esta separación también facilita agregar una construcción como `match`: la gramática puede reconocer el punto donde comienza una expresión, y el Pratt parser puede consumir completamente la forma `match (scrutinee) { pattern => body, ... }` sin forzar una reestructuración global de la gramática.

Comparado con un parser de precedencia implementado con muchas funciones manuales, el Pratt parser evita duplicar lógica para cada nivel de precedencia. Comparado con parser combinators, mantiene un modelo más directo y predecible para un compilador académico. Comparado con una solución LR externa, permite preservar el control sobre los errores y sobre la construcción inmediata del AST.

3.4. AST como frontera entre sintaxis y semántica

El AST es la primera representación estable del lenguaje. No debe confundirse con el CST. El CST conserva detalles de la derivación sintáctica, incluyendo símbolos auxiliares, delimitadores y decisiones gramaticales. El AST, en cambio, conserva conceptos del lenguaje: declaraciones, expresiones, tipos, miembros, patrones y cuerpos.

Esta separación evita que la fase semántica dependa de detalles accidentales de la gramática. Por ejemplo, el parser puede reconocer una función inline y una función con bloque mediante reglas distintas, pero la semántica solo necesita una declaración de función con nombre, parámetros, tipo de retorno opcional y cuerpo. De forma similar, la sintaxis concreta de un acceso, una llamada o una indexación se transforma en nodos explícitos que la semántica puede analizar sin razonar sobre tokens.

La extensión `match` muestra claramente el papel del AST. Una entrada como:

Listing 3: Expresión `match` en sintaxis fuente

```
match (s) {  
  Circle as c => c.area(),  
  _ => 0  
}
```

no se conserva como una lista de tokens. En el AST aparece como una expresión `Match` con un `scrutinee` y una lista de brazos. Cada brazo contiene un patrón y un cuerpo. Los patrones también tienen estructura propia: wildcard, literales, bindings y patrones de tipo con binding. Esta representación es necesaria para que la semántica pueda introducir variables locales, validar tipos, calcular el tipo resultante del `match` y reportar errores como falta de exhaustividad.

Por tanto, agregar `match` no fue solamente agregar una palabra clave al lexer ni una regla al parser. La construcción atraviesa el frontend completo: tokenización, análisis sintáctico, AST y contrato con la semántica.

3.5. Separación de categorías de error

El frontend distingue errores léxicos de errores sintácticos. Esta decisión es pequeña, pero importante. Un carácter inesperado, un string sin cerrar o un token imposible pertenecen al nivel léxico. En cambio, una secuencia de tokens válida que no forma una construcción del lenguaje pertenece al nivel sintáctico. Separar ambas categorías ayuda a depurar, permite salidas más precisas y evita que una fase posterior intente interpretar un programa que ni siquiera fue reconocido correctamente.

4. Análisis semántico e HIR

Después del frontend, el compilador conoce la estructura del programa, pero todavía no sabe si esa estructura tiene sentido. La fase semántica responde preguntas que la gramática no puede resolver: si una variable está definida, si una llamada tiene la aridad correcta, si un tipo existe, si una expresión usada como condición es booleana, si un método pertenece al receptor, si un cast es admisible, si una redefinición respeta la firma del padre o si un tipo satisface un protocolo.

Esta fase es el centro de las garantías estáticas del compilador. Un parser puede aceptar un programa con un método inexistente o con una operación aritmética sobre strings; la semántica debe rechazarlo. Por eso, el sistema no se limita a construir árboles para programas correctos, sino que implementa diagnósticos diferenciados para programas inválidos.

4.1. Registro de tipos y análisis en varias pasadas

El sistema usa un registro de tipos para almacenar información sobre tipos nominales, protocolos, atributos, métodos, constructores, padres y firmas. Esta decisión responde a una necesidad del lenguaje: las declaraciones globales pueden referirse entre sí, las funciones pueden ser recursivas, los tipos pueden mencionar otros tipos y la herencia requiere conocer la jerarquía completa.

Una alternativa sería analizar el programa en una sola pasada, registrando y validando todo en el orden en que aparece. Ese enfoque simplifica la implementación inicial, pero impone restricciones artificiales: dificultaría llamadas a funciones declaradas después, complicaría la recursión y haría más frágil la validación de herencia. En cambio, una estrategia de varias pasadas permite separar la recolección de nombres de la validación de detalles.

Primero se registran los nombres disponibles; luego se completan firmas, atributos y métodos; después se valida la herencia; finalmente se analizan cuerpos y expresiones. Esta organización es más robusta para un lenguaje con funciones globales, objetos, protocolos y type inference.

4.2. Inferencia iterativa

HULK permite omitir anotaciones de tipo en varios contextos. Por eso, el compilador no puede limitarse a comprobar anotaciones explícitas: también debe inferir tipos de expresiones, variables, parámetros y retornos cuando sea posible. La dificultad es que la inferencia no siempre es local. El tipo de un parámetro puede deducirse por cómo se usa en el cuerpo de una función, pero también por los sitios de llamada donde se invoca esa función.

El sistema implementa una estrategia iterativa. En cada pasada se conservan firmas inferidas de funciones, métodos y constructores. Si una firma cambia, se repite el análisis; cuando no aparecen cambios nuevos, se ejecuta una pasada final que reporta errores de inferencia no resueltos. Esta decisión permite resolver más programas que una inferencia puramente local y, al mismo tiempo, evita intentar implementar un sistema de inferencia demasiado ambicioso.

Existen varias alternativas. Una primera opción sería exigir anotaciones en todos los parámetros y retornos. Eso simplificaría la semántica, pero se alejaría del diseño de HULK, donde las anotaciones son opcionales en muchos lugares. Una segunda opción sería inferir solo desde literales y operadores locales; esto funcionaría para casos simples, pero fallaría ante funciones llamadas desde distintos contextos. Una tercera opción sería intentar una inferencia general al estilo ML. Esa alternativa no es adecuada para este proyecto porque HULK combina subtipado nominal, objetos, protocolos, métodos virtuales y casts dinámicos, lo cual vuelve el problema más complejo.

La inferencia iterativa es, por tanto, un compromiso razonable: aumenta expresividad sin convertir el compilador en un sistema de inferencia completo de propósito general.

4.3. HIR tipada como contrato interno

La HIR es una representación de alto nivel, pero con decisiones semánticas ya resueltas. Cada expresión de HIR tiene un identificador, un span, un tipo y una variante de expresión. Las variables se resuelven a símbolos; las llamadas se clasifican como builtins, funciones globales, métodos o functors; los accesos a atributos se resuelven; y las llamadas a métodos incluyen información de despacho.

La HIR existe para evitar que el lowering repita el trabajo de la semántica. Si el compilador bajara directamente desde el AST, la fase de lowering tendría que volver a buscar nombres, resolver métodos, determinar tipos y decidir si una llamada es estática, virtual o a `base`. Eso acoplaría dos responsabilidades distintas. Con una HIR tipada, el lowering puede asumir que el programa ya fue validado y concentrarse en convertir construcciones de alto nivel en operaciones de IR.

Esta frontera también es útil para `match`. La HIR permite transformar la construcción en una combinación de `let`, `if`, `is`, `as` y comparaciones. Así, la semántica preserva el significado de `match` sin obligar a que la IR o el backend conozcan una instrucción especial para patrones.

4.4. Chequeo de objetos, herencia y protocolos

En el modelo de objetos, la semántica verifica que los tipos existan, que los atributos no se dupliquen, que los métodos no se dupliquen dentro del mismo tipo, que no existan ciclos de herencia y que no se herede de tipos primitivos como `Number`, `String` o `Boolean`. También verifica que un método que redefine a otro mantenga una firma compatible con la del padre.

Para protocolos, el sistema valida conformidad estructural: un tipo puede usarse donde se espera un protocolo si posee los métodos requeridos con aridad y tipos compatibles. Esto introduce una forma de polimorfismo basada en comportamiento. La verificación no se reduce al nombre del método; también se revisan parámetros y retornos, respetando reglas de varianza cuando corresponden.

Esta decisión conecta Lenguajes de Programación con Compilación. Desde el punto de vista del lenguaje, los protocolos permiten escribir código más abstracto. Desde el punto de vista

del compilador, obligan a registrar firmas, comprobar conformidad y materializar llamadas sobre valores cuyo tipo estático puede ser protocolario.

4.5. Errores semánticos diferenciados

Una fortaleza de la fase semántica es que distingue errores de naturaleza diferente. Entre los casos cubiertos aparecen funciones duplicadas, parámetros duplicados, símbolos indefinidos, métodos indefinidos, objetivos de asignación inválidos, errores de aridad, tipos desconocidos, incompatibilidades de tipos, operandos inválidos, condiciones no booleanas, retornos incorrectos, argumentos incorrectos, inferencia fallida, herencia circular, métodos faltantes de protocolos, acceso a atributos privados, herencia desde primitivos, firmas incompatibles en override, errores de conformidad protocolaria y expresiones `match` no exhaustivas.

Esta variedad de errores demuestra que el compilador no solo acepta ejemplos correctos. También rechaza programas inválidos con diagnósticos asociados a la fase correcta. En una defensa académica esto es esencial: un compilador no se evalúa únicamente por los programas que ejecuta, sino también por los programas inválidos que detecta antes de generar código.

5. Representación intermedia y lowering

5.1. Motivación de una IR propia

Una decisión central del sistema es no emitir LLVM directamente desde la HIR. Aunque LLVM es un backend potente, su nivel de abstracción no coincide con las necesidades inmediatas de HULK. La HIR todavía contiene decisiones de lenguaje: objetos, métodos, acceso a atributos, vectores, closures, casts, type tests y valores expresivos. LLVM, en cambio, exige detalles más concretos sobre funciones, punteros, layouts, llamadas externas y tipos de bajo nivel.

La IR propia funciona como puente entre ambos mundos. Conserva suficiente información de HULK para expresar objetos, funciones, métodos, vectores, closures y llamadas dinámicas de forma legible; pero elimina la sintaxis de alto nivel y hace explícito el control de flujo mediante etiquetas, saltos y ramas. Esta representación es más cercana a código de tres direcciones y facilita pruebas golden, depuración y razonamiento sobre lowering.

Una alternativa habría sido bajar desde AST o HIR directamente a LLVM. Esa opción reduce el número de fases, pero aumenta el acoplamiento: cada cambio semántico obligaría a tocar el backend LLVM. Otra alternativa habría sido implementar una máquina virtual propia y ejecutar la IR directamente. Ese camino es válido, pero el sistema opta por LLVM como backend principal y usa la IR como una etapa de normalización antes de la emisión.

5.2. Relación con BANNER

La IR del proyecto está inspirada en la filosofía de BANNER, la representación intermedia didáctica propuesta para HULK. BANNER organiza los programas en secciones de tipos, datos y código, y promueve una forma explícita de representar temporales, control de flujo, llamadas, objetos y acceso a memoria. Esa idea se conserva en la Hulk IR mediante secciones `.TYPES`, `.DATA` y `.CODE`.

Sin embargo, la IR implementada no pretende ser BANNER puro. BANNER se presenta como una IR minimalista con una filosofía cercana a “todo es un número”. La Hulk IR conserva tipos de IR más ricos, porque debe servir de contrato con un backend LLVM que distingue números, booleanos, strings, objetos, vectores, iterables y functors. Esta adaptación permite mantener la claridad de una IR de tres direcciones sin perder información útil para la emisión LLVM.

5.3. Forma general de la IR

Un programa IR contiene layouts lógicos de tipos, datos estáticos y funciones. Textualmente se imprime en tres secciones:

Listing 4: Estructura textual de la IR

```
.TYPES
type Point #0 {
  attr #0 x: Number
  attr #1 y: Number
  method #0 getX : Point_getX
}

.DATA
data @s0 = "hello"

.CODE
entry #0
function entry #0 -> Object {
  ...
}
```

La sección `.TYPES` describe atributos y métodos de cada tipo. La sección `.DATA` almacena valores estáticos, especialmente strings. La sección `.CODE` contiene funciones globales, métodos, lambdas y la función de entrada.

5.4. Instrucciones principales

La IR usa valores de lectura y lugares de escritura. Los valores incluyen temporales, locales, parámetros, constantes, referencias a datos, `null` y `unit`. Los destinos de escritura incluyen

temporales y locales.

Entre las instrucciones principales aparecen movimiento, operaciones unarias y binarias, etiquetas, saltos, ramas, retornos, llamadas, llamadas estáticas, llamadas virtuales, llamadas a **base**, asignación de objetos, lectura y escritura de atributos, creación y acceso a vectores, creación de closures, llamadas a closures, lectura de capturas, pruebas de tipo y casts.

Grupo	Instrucciones representativas
Control de flujo	Label, Jump, Branch, Return.
Llamadas	Call, StaticCall, VirtualCall, BaseCall.
Objetos	Allocate, GetAttr, SetAttr.
Vectores	NewVector, VectorLen, VectorPush, VectorGet, VectorSet.
Closures	MakeClosure, ClosureCall, GetCapture.
Tipos dinámicos	TypeTest, TypeCast.

El hecho de que **match** no aparezca como instrucción propia es deliberado. La construcción se resuelve antes mediante desugaring. Esto mantiene más pequeño el conjunto de instrucciones y evita duplicar lógica en lowering, IR y backend.

5.5. Lowering de objetos y métodos

El lowering crea layouts de tipos, hereda atributos del padre y conserva slots de métodos. Si un método redefine uno del padre, ocupa el mismo slot y actualiza la función destino. Esta decisión prepara el despacho dinámico mediante vtables.

La razón para usar slots estables es que una llamada virtual no debe depender del nombre textual del método en tiempo de ejecución. El compilador puede determinar en tiempo de compilación qué posición ocupa un método dentro de la tabla virtual. Luego, en runtime, basta con obtener la vtable del objeto y llamar la función almacenada en ese slot.

El **self** implícito de los métodos se convierte en un parámetro explícito en IR. Esta transformación es común en compiladores de lenguajes orientados a objetos: el método pasa a ser una función ordinaria que recibe como primer argumento el objeto receptor. De esta forma, el backend puede emitir llamadas más uniformes.

5.6. Lowering de base

La llamada **base(...)** representa una decisión semántica particular: no debe buscar dinámicamente el método del receptor, sino invocar la implementación del padre. Por eso se resuelve antes del backend. En IR aparece como **BaseCall**, con información suficiente para emitir una llamada directa o semidirecta al método correspondiente del ancestro.

Esta decisión evita confundir **base** con despacho virtual ordinario. Si **base** se resolviera como una llamada virtual sobre el receptor, podría terminar llamando otra vez al override del hijo.

Al convertirla en una operación especial de IR, el compilador preserva la semántica esperada de herencia.

5.7. Lowering de vectores, rangos y ciclos

Los vectores literales se convierten en operaciones explícitas de construcción y carga de elementos. La indexación se baja como `VectorGet`; la asignación por índice se baja como `VectorSet`; y la longitud se representa como `VectorLen`. Esta decisión evita que el backend tenga que reconocer sintaxis de vectores; solo emite llamadas o instrucciones asociadas a operaciones ya normalizadas.

Los ciclos `for` se relacionan con el protocolo `Iterable`. Conceptualmente, un `for` puede desazucararse a una estructura basada en `next()` y `current()`. Esta transformación conecta una construcción de control de alto nivel con llamadas a métodos conocidas. En runtime, `range` produce un objeto iterable con `vtable` propia.

5.8. Lowering de lambdas y closures

Las lambdas se bajan a funciones separadas. Cuando una lambda captura variables del entorno, el lowering genera una closure que almacena el puntero a la función y los valores capturados. La instrucción `MakeClosure` representa la creación de ese objeto funcional y `ClosureCall` representa la invocación posterior.

Esta estrategia separa el código de la lambda del entorno capturado. Es una técnica usual en compiladores con funciones anónimas: el cuerpo de la lambda se convierte en una función regular, mientras que el entorno se materializa como una estructura de datos accesible desde esa función. La instrucción `GetCapture` hace explícita la lectura de variables capturadas.

6. Backend LLVM y runtime

6.1. Motivación de LLVM como backend

El backend elegido es LLVM. Esta decisión evita implementar directamente generación de ensamblador o código máquina para una arquitectura específica. LLVM IR funciona como un lenguaje intermedio de bajo nivel, ampliamente usado para optimización y generación de código nativo. En el sistema implementado, el compilador emite LLVM IR textual y delega a `clang` la producción del ejecutable final.

La alternativa de generar ensamblador manualmente habría aumentado considerablemente el alcance del backend: habría que manejar convenciones de llamada, layouts concretos, registros, pila, alineación y detalles de plataforma. La alternativa de construir una VM propia habría permitido controlar la ejecución, pero implicaría diseñar instrucción, pila, heap, dispatch, GC o administración de memoria, y un ciclo de interpretación. LLVM permite concentrar el

trabajo en la traducción semántica de HULK y reutilizar una infraestructura existente para la parte final de código nativo.

6.2. Emisión de LLVM IR

El crate de generación LLVM convierte un `IrProgram` en un módulo textual de LLVM. Declara funciones externas del runtime para impresión, matemáticas, strings, objetos, vtables, type tests, casts, rangos, vectores y closures. Luego emite datos estáticos, vtables, funciones del programa y una función `main` compatible con C que llama a la función de entrada de HULK.

El mapeo general de tipos es el siguiente:

Tipo IR	Tipo LLVM	Comentario
Number	double	Operaciones aritméticas y comparaciones numéricas.
Boolean	i1 / i8	i1 para lógica interna; i8 en fronteras con C cuando hace falta.
String	ptr	Puntero a descriptor de string del runtime.
Object y tipos de usuario	ptr	Punteros a objetos con vtable.
Vector(T)	ptr	Puntero a estructura <code>HulkVector</code> .
Iterable(T)	ptr	Objeto iterable con vtable.
Functor	ptr	Closure o valor invocable.

6.3. Papel del runtime en C

El runtime en C implementa operaciones que requieren soporte de ejecución: impresión de valores, funciones matemáticas, manipulación de strings, reserva de objetos, acceso a atributos, vtables, type tests, casts, vectores, rangos, closures y errores dinámicos.

Emitir todas estas operaciones directamente como LLVM textual habría hecho crecer mucho el backend. Por ejemplo, concatenar strings, redimensionar vectores, comprobar límites de índice o construir una closure son operaciones con detalles de memoria y llamadas auxiliares. Al moverlas al runtime, el LLVM generado puede limitarse a invocar funciones externas bien definidas.

Esta decisión también mejora la separación de responsabilidades. El backend se encarga de traducir IR a llamadas LLVM correctas; el runtime se encarga de implementar el comportamiento operativo de esas llamadas. Así, una mejora en la representación de strings o vectores puede hacerse en C sin reescribir toda la emisión LLVM.

6.4. El runtime no es una máquina virtual

Es importante distinguir el runtime de una VM. El runtime no interpreta instrucciones HULK ni ejecuta la IR propia. Su función es proveer operaciones auxiliares enlazadas al programa

generado. El backend real del sistema es LLVM: el compilador emite LLVM IR y `clang` produce un binario nativo. El runtime se enlaza como biblioteca de soporte, de forma similar a como otros compiladores enlazan bibliotecas estándar o rutinas de ejecución.

Por tanto, el sistema no presenta dos backends alternativos. Presenta un backend LLVM complementado por una biblioteca C que implementa operaciones de runtime.

6.5. Driver y modos de inspección

El driver ofrece dos tipos de uso. Sin flags, compila el programa y produce un ejecutable. Con flags, imprime representaciones intermedias útiles para depuración y defensa académica: AST, resultado del chequeo semántico, HIR, IR y LLVM IR.

Listing 5: Uso esperado del driver

```
hulkc programa.hulk          # compila y produce ./output
hulkc programa.hulk --ast     # muestra AST
hulkc programa.hulk --check   # ejecuta chequeo semántico
hulkc programa.hulk --hir     # muestra HIR
hulkc programa.hulk --ir      # muestra Hulk IR
hulkc programa.hulk --emit-llvm # muestra LLVM IR
```

Estos modos muestran que el compilador puede inspeccionarse por capas. Esto es especialmente útil para probar extensiones: una construcción como `match` debe verse en AST, debe validarse en semántica, debe transformarse en HIR y no debe requerir una instrucción especial en IR.

7. Extensión propia de lenguaje: match

7.1. Problema de diseño

El lenguaje HULK ya ofrece condicionales, pruebas dinámicas de tipo mediante `is` y casts mediante `as`. Con esas herramientas, es posible distinguir casos de forma manual. Sin embargo, cuando un programa recibe valores de un tipo general y necesita actuar según su valor o tipo dinámico, el código se vuelve repetitivo.

Listing 6: Clasificación manual sin match

```
if (s is Circle) {
    let c = s as Circle in c.area();
} elif (s is Rectangle) {
    let r = s as Rectangle in r.area();
} else {
    0;
};
```

Este estilo tiene tres problemas. Primero, repite el scrutinee `s` en cada rama. Segundo, separa la prueba de tipo del binding resultante. Tercero, obliga al programador a recordar manualmente que después de `s is Circle` debe usar `s as Circle` para obtener un valor con tipo estático más específico. En programas con muchas alternativas, la lógica de clasificación queda dispersa.

La construcción `match` responde a ese problema. Permite agrupar en una única expresión la evaluación del valor, la selección del caso, la introducción de bindings y el resultado de cada brazo.

Listing 7: Código equivalente con `match`

```
match (s) {  
  Circle as c => c.area(),  
  Rectangle as r => r.area(),  
  _ => 0  
};
```

7.2. Alternativas consideradas

La primera alternativa era no agregar ninguna construcción nueva y depender únicamente de `if`, `is` y `as`. Esta opción tenía el menor costo de implementación, pero no aportaba una extensión de lenguaje clara. Además, dejaba en manos del programador la coordinación entre prueba dinámica, cast y binding.

La segunda alternativa era agregar un `switch` limitado a literales. Esa opción habría resuelto casos simples sobre números, strings o booleanos, pero no habría aportado una mejora importante para el modelo de objetos de HULK. En un lenguaje con herencia y type tests, la clasificación por tipo dinámico es tan importante como la clasificación por valor literal.

La tercera alternativa era introducir tipos algebraicos cerrados, como los `enum` de Rust o los tipos suma de ML/Haskell, y diseñar un pattern matching completo sobre variantes. Esa opción es expresiva, pero demasiado grande para el alcance del sistema: implicaría agregar nuevas formas de declarar tipos, constructores de variantes, reglas de exhaustividad profunda y posiblemente cambios importantes en representación de valores.

La decisión adoptada fue incorporar un `match` más pequeño, compatible con el modelo de objetos existente: patrones literales, wildcard, bindings, patrones de tipo y patrones de tipo con binding. Esta variante no pretende competir con el pattern matching completo de lenguajes funcionales, pero sí resuelve un problema real de expresividad en HULK.

7.3. Sintaxis y semántica básica

La forma general de `match` es:

Listing 8: Forma general de `match`

```
match (scrutinee) {  
  pattern_1 => expr_1,
```

```

    pattern_2 => expr_2,
    _ => default_expr
}

```

El scrutinee se evalúa una sola vez. Luego los brazos se prueban de arriba hacia abajo. El primer patrón que coincide determina el valor resultante de la expresión completa. Como HULK es un lenguaje basado en expresiones, cada cuerpo de brazo produce un valor y el `match` completo también produce un valor.

La evaluación única del scrutinee es importante. Si el scrutinee tuviera efectos secundarios, reevaluarlo por cada brazo cambiaría el significado del programa. Por eso el desugaring conceptual guarda el resultado en una variable sintética interna antes de probar los patrones.

7.4. Patrones soportados

El sistema implementa cinco familias de patrones:

Patrón	Sintaxis	Significado
Wildcard	<code>_</code>	Coincide siempre y no introduce variables.
Literal	<code>42, "hola", true</code>	Coincide si el scrutinee es igual al literal.
Binding	<code>x</code>	Coincide siempre e introduce una variable local con el valor del scrutinee.
Tipo	<code>Circle</code>	Coincide si el scrutinee satisface <code>scrutinee is Circle</code> .
Tipo con binding	<code>Circle as c</code>	Coincide si el scrutinee es <code>Circle</code> e introduce <code>c</code> con tipo refinado.

La distinción entre binding y patrón de tipo se apoya en la convención de nombres del lenguaje: los tipos usan nombres con inicial mayúscula, mientras que las variables suelen usar inicial minúscula. Esta decisión evita introducir una palabra adicional como `case` para cada brazo y mantiene la sintaxis compacta.

7.5. Narrowing de tipos

El caso más importante desde el punto de vista del sistema de tipos es `Type as x`. Este patrón combina tres acciones. Primero, realiza una prueba dinámica de tipo mediante `is`. Segundo, si la prueba tiene éxito, realiza un cast seguro mediante `as`. Tercero, introduce una variable local cuyo tipo estático es más específico que el del scrutinee original.

Listing 9: Narrowing mediante patrón de tipo

```

function areaOf(s: Shape): Number =>
    match (s) {
        Circle as c => c.radius() * c.radius() * 3,
    }

```

```
Rectangle as r => r.width() * r.height(),  
_ => 0  
};
```

Dentro del primer brazo, `c` tiene tipo `Circle`; dentro del segundo, `r` tiene tipo `Rectangle`. Esto evita casts manuales y reduce la posibilidad de usar una variable con un tipo demasiado general. La construcción se parece a los smart casts de lenguajes modernos, pero en este diseño el narrowing se hace explícito mediante el binding del patrón.

7.6. Exhaustividad conservadora

Todo `match` debe tener un brazo catch-all, escrito como wildcard `_` o como binding. Si no existe tal brazo, la semántica reporta un error de match no exhaustivo. Esta regla es conservadora: no intenta demostrar que una lista de patrones de tipo cubre todos los casos posibles.

La razón es que HULK trabaja con jerarquías nominales de objetos y no con tipos suma cerrados. En un lenguaje con enums algebraicos, el compilador conoce todas las variantes posibles de un tipo y puede verificar exhaustividad de forma profunda. En HULK, una jerarquía como `Shape`, `Circle` y `Rectangle` no necesariamente está cerrada. Puede existir otro subtipo de `Shape`, o puede agregarse más adelante dentro del mismo programa.

Por tanto, exigir un caso general es una decisión segura. La regla no es tan precisa como el chequeo exhaustivo de Rust sobre enums, pero evita asumir que el compilador conoce todos los subtipos posibles. Esta decisión también mantiene simple el desugaring: siempre existe una rama final que produce resultado si ningún patrón anterior coincide.

7.7. Desugaring a construcciones existentes

Una decisión clave fue no agregar una instrucción `Match` a la IR. En lugar de eso, `match` se transforma durante la construcción de HIR en una combinación de primitivas ya existentes: `let`, `if`, `is`, `as` y comparaciones de igualdad.

Listing 10: Esquema conceptual de desugaring

```
let _match_scr_N = scrutinee in  
  if (_match_scr_N is Circle)  
    let c = _match_scr_N as Circle in body_circle  
  elif (_match_scr_N == literal)  
    body_literal  
  else  
    body_default
```

Esta estrategia tiene varias ventajas. Primero, reduce el tamaño de la IR. Segundo, reutiliza reglas semánticas ya existentes para condicionales, bindings, type tests y casts. Tercero, mantiene el backend LLVM independiente de una construcción de alto nivel. Cuarto, facilita

la validación: si `if`, `let`, `is` y `as` ya funcionan correctamente, `match` puede apoyarse en ellos después de verificar sus reglas propias.

Desde el punto de vista de diseño de compiladores, esta es una forma de separar azúcar sintáctica expresiva de primitivas semánticas mínimas. `match` mejora la expresividad del lenguaje fuente, pero no obliga a multiplicar la complejidad en las fases de bajo nivel.

7.8. Comparación con otros lenguajes

El `match` de HULK se inspira en mecanismos presentes en varios lenguajes, pero adopta un alcance más pequeño y coherente con el proyecto.

En Rust, `match` es una construcción central del lenguaje. Su mayor fortaleza es la exhaustividad sobre tipos algebraicos, especialmente `enum`. Cuando un `enum` tiene variantes cerradas, el compilador puede exigir que todos los casos estén cubiertos. Además, Rust permite patrones estructurados, destructuring, guards, bindings y combinaciones más expresivas. HULK toma de Rust la idea de selección por patrones, pero no adopta su modelo de ADTs ni su exhaustividad profunda. La diferencia responde al modelo de tipos: Rust puede razonar sobre variantes cerradas; HULK trabaja principalmente con objetos y herencia abierta.

En Swift, `switch` funciona como una construcción de pattern matching rica. Puede trabajar sobre `enums`, tuplas, rangos, bindings y condiciones adicionales. Swift combina el pattern matching con una sintaxis expresiva para casos complejos. La versión de HULK es más limitada: no hay patrones de tuplas, rangos ni destructuring de objetos. Sin embargo, comparte con Swift la idea de que una construcción de selección puede ser más general que un `switch` tradicional sobre constantes.

En Kotlin, la construcción comparable es `when`. Kotlin permite distinguir casos por valor y por tipo, y su sistema de smart casts permite tratar una variable como un tipo más específico después de una comprobación. HULK se acerca a Kotlin en este punto: el patrón `Circle as c` combina prueba de tipo y uso refinado dentro de la rama. La diferencia es que Kotlin puede refinar la misma variable después de ciertas condiciones, mientras que HULK introduce explícitamente un nuevo binding refinado.

En Scala, `match` se integra con case classes, extractores y un sistema de patrones más estructural. Esto permite descomponer valores complejos, no solo clasificarlos por tipo. HULK no implementa extractores ni destructuring de atributos, porque eso habría requerido reglas adicionales de privacidad, visibilidad de campos y representación de patrones anidados. El diseño elegido se mantiene en un nivel compatible con los mecanismos ya existentes del lenguaje.

En OCaml y Haskell, el pattern matching está asociado a tipos algebraicos y programación funcional. Allí los patrones son una forma natural de consumir valores contruidos por variantes. HULK no introduce un paradigma funcional completo ni ADTs cerrados; por eso su `match` debe entenderse como una extensión pragmática para un lenguaje de objetos, no como una adopción completa del modelo ML.

Esta comparación muestra que el valor de `match` en HULK no está en igualar la expresividad

de Rust, Scala o Haskell. Su valor está en agregar una construcción pequeña, comprensible y útil para clasificar valores en un lenguaje con objetos, casts y type tests, manteniendo bajo el costo de implementación mediante desugaring.

8. Mecanismos avanzados de HULK integrados

8.1. Protocolos estructurales

Los protocolos permiten expresar comportamiento esperado sin depender de una clase concreta. Un protocolo contiene firmas de métodos, y un tipo conforma al protocolo si tiene métodos compatibles. Esta forma de conformidad es estructural: no es obligatorio declarar explícitamente una relación nominal como `implements`.

Listing 11: Protocolo como contrato de comportamiento

```
protocol Drawable {
    draw(): Object;
}

type Circle() {
    draw(): Object => print("circle");
}

let x: Drawable = new Circle() in x.draw();
```

Esta decisión reduce boilerplate y permite definir abstracciones después de haber escrito tipos concretos. Se parece a Go, donde una interfaz se satisface implícitamente si el tipo ofrece los métodos requeridos. También se parece a TypeScript, donde la compatibilidad se basa en la estructura de los miembros disponibles. En contraste, Java y C# usan interfaces nominales: una clase debe declarar explícitamente que implementa una interfaz. Rust, por su parte, usa traits, pero exige bloques `impl` explícitos.

La ventaja de la conformidad estructural es la flexibilidad. La desventaja es la posibilidad de conformidad accidental: un tipo puede satisfacer un protocolo por coincidencia de nombres y firmas, aunque el programador no lo haya pensado como una implementación conceptual de esa abstracción. El compilador mitiga parcialmente este riesgo verificando aridad y tipos, pero no puede verificar intención semántica.

8.2. interface como alias de protocol

El sistema acepta `interface` como alias sintáctico de `protocol`. Esta decisión no introduce una abstracción nueva: ambos términos representan una colección de firmas de métodos. Su valor está en familiaridad. Programadores provenientes de Java, C#, TypeScript o Go reconocen rápidamente la palabra `interface`. Al mismo tiempo, conservar `protocol` mantiene compatibilidad con la terminología de HULK.

El alias es deliberadamente superficial. No existen dos mecanismos distintos de conformidad ni dos jerarquías separadas. La semántica interna debe normalizar ambos casos hacia la misma representación de protocolo. Esta decisión evita duplicar reglas y mantiene simple el análisis semántico.

8.3. Varianza en firmas de protocolos

La conformidad protocolaria no debe limitarse a comparar textos de firmas. Para que un tipo sea sustituible donde se espera un protocolo, sus métodos deben respetar reglas de compatibilidad. En particular, los retornos pueden ser covariantes y los parámetros contravariantes.

La covarianza en retornos significa que una implementación puede devolver un tipo más específico que el esperado por el protocolo. Si un protocolo exige un método que devuelve `Animal`, una implementación que devuelve `Dog` es segura, porque todo `Dog` puede usarse como `Animal`. La contravarianza en parámetros significa que una implementación puede aceptar un tipo más general que el exigido. Si un protocolo invocará un método con valores de tipo `Dog`, una implementación que acepta `Animal` también puede recibir esos argumentos.

Estas reglas reflejan el principio de sustitución: un valor que conforma a un protocolo debe poder usarse sin romper las expectativas de quien invoca sus métodos. Desde el punto de vista del compilador, esto exige comparar tipos mediante la relación de conformidad, no solo mediante igualdad exacta.

8.4. Vectores, arreglos e iterables

Los vectores agregan una estructura homogénea de colección. El sistema soporta literales, indexación, asignación por índice, longitud, generadores y construcción dimensionada. Además, los vectores se integran con la noción de iterabilidad, por lo que pueden recorrerse con `for`.

Esta decisión conecta tres niveles. En el lenguaje fuente, el usuario trabaja con una sintaxis compacta para colecciones. En semántica, el compilador debe inferir o verificar el tipo de los elementos y comprobar que los índices sean numéricos. En IR y runtime, los vectores se representan mediante operaciones explícitas de creación, acceso, actualización y recorrido.

El protocolo `Iterable` sirve como contrato para ciclos `for`. Conceptualmente, un ciclo `for` puede traducirse a una combinación de `next()` y `current()`. Esta transformación evita que `for` sea una construcción mágica del backend: se apoya en un comportamiento protocolario reconocible.

8.5. Lambdas, functors y closures

Las lambdas permiten introducir funciones anónimas. Los functors permiten tratar valores invocables como objetos de primer orden. Las closures extienden esa idea al permitir que una lambda capture variables del entorno donde fue creada.

Desde el punto de vista de Lenguajes de Programación, esto acerca HULK a un estilo multiparadigma: objetos, funciones globales y funciones de orden superior pueden convivir. Desde el punto de vista de Compilación, las closures obligan a separar código y entorno. El código de la lambda puede bajarse a una función sintética, pero las variables capturadas deben almacenarse en una estructura accesible durante la ejecución.

El lowering refleja esta separación mediante instrucciones de creación de closures, almacenamiento de capturas y llamadas indirectas. El runtime provee la estructura concreta para representar el puntero a función y el arreglo de capturas.

9. Validación y evidencia experimental

9.1. Estrategia de pruebas por capas

La validación del compilador se apoya en una estrategia por capas. Esta estrategia es coherente con la arquitectura del sistema: cada fase tiene responsabilidades diferentes y, por tanto, debe probarse con criterios diferentes.

Las pruebas de lexer y parser validan que el lenguaje fuente se reconoce correctamente. Las pruebas de frontend verifican que la estructura AST producida corresponde a la intención sintáctica. Las pruebas semánticas comprueban programas válidos e inválidos. Los golden tests de IR estabilizan el formato de lowering. Las pruebas de backend verifican emisión LLVM y llamadas al runtime. Las pruebas end-to-end comprueban que un programa HULK puede compilarse, enlazarse y ejecutarse.

Nivel	Riesgo que valida
Lexer/parser	Que el texto fuente se tokenice y estructure según la sintaxis esperada.
AST/frontend	Que la sintaxis concreta se convierta en nodos abstractos correctos.
Semántica	Que se acepten programas correctos y se rechacen errores de nombres, tipos, herencia y protocolos.
HIR/lowering	Que las decisiones semánticas se transformen en IR sin perder significado.
Backend LLVM	Que la IR pueda emitirse como LLVM textual válido.
End-to-end	Que el binario final produzca la salida esperada o el error dinámico correspondiente.

9.2. Validación de match

La extensión `match` requiere pruebas específicas porque atraviesa varias fases. No basta con verificar que el parser acepte la palabra clave. Debe comprobarse que los patrones se representen en AST, que los bindings se introduzcan en el scope correcto, que los patrones de tipo produzcan narrowing, que los cuerpos de los brazos se unifiquen a un tipo resultante y que la falta de catch-all sea reportada como error semántico.

Los casos de prueba relevantes incluyen wildcard, literales numéricos, literales booleanos, literales string, bindings, patrones de tipo, patrones de tipo con binding, múltiples brazos, mezcla de patrones y detección de matches no exhaustivos. También es importante probar que el scrutinee no se reevalúa por cada brazo y que el binding refinado solo existe dentro del cuerpo correspondiente.

9.3. Validación del backend y runtime

Las pruebas de backend deben cubrir aritmética, booleanos, comparaciones, strings, concatenación, funciones globales, recursión, ciclos, vectores, rangos, objetos, herencia, despacho dinámico, **base**, closures y errores dinámicos. En particular, son importantes los casos donde el LLVM generado llama al runtime: impresión, strings, vectores, casts, type tests, rangos y closures.

Los errores dinámicos también forman parte de la validación. División por cero, módulo por cero, casts inválidos, llamadas sobre objetos nulos o accesos fuera de rango deben producir fallos controlados, no comportamiento indefinido silencioso.

9.4. Comandos de validación

El repositorio provee comandos de construcción, prueba e inspección. La validación general del workspace puede ejecutarse con `cargo test`. La construcción del binario puede realizarse con `cargo build --release -p hulk-driver` o mediante `make build`. La prueba integrada puede invocarse mediante `make test`, que construye el compilador y ejecuta la suite externa configurada.

Listing 12: Comandos principales de validación

```
cargo test --workspace --all-targets --locked
cargo build --release -p hulk-driver
make build
make test
./hulk examples/demo.hulk
./output
./hulk examples/demo.hulk --ast
./hulk examples/demo.hulk --check
./hulk examples/demo.hulk --hir
./hulk examples/demo.hulk --ir
./hulk examples/demo.hulk --emit-llvm
```

Estos comandos permiten comprobar compilación del workspace, construcción del driver, ejecución de programas HULK y visualización de las principales representaciones intermedias.

10. Limitaciones y trabajo futuro

10.1. Gestión de memoria

El runtime usa una estrategia de administración de memoria simplificada, basada en reserva sin liberación individual durante la ejecución de los programas de prueba. Esta decisión reduce la complejidad del runtime y es razonable para un compilador académico, pero no equivale a un recolector de basura completo.

Una evolución natural sería implementar un GC o una estrategia más robusta de administración de memoria. Esto sería especialmente importante para programas largos, creación intensiva de objetos, strings, vectores y closures. La decisión actual prioriza claridad del backend y funcionalidad observable sobre completitud del sistema de memoria.

10.2. Exhaustividad de match

El chequeo de exhaustividad de `match` es conservador. El compilador exige un brazo catch-all, pero no intenta demostrar cobertura completa sobre jerarquías de tipos. Esta decisión se debe a que HULK no incorpora tipos algebraicos cerrados ni clases selladas.

Como trabajo futuro, se podría extender el lenguaje con jerarquías cerradas o tipos suma. En ese caso, el compilador podría verificar que todos los casos de un conjunto cerrado están cubiertos sin exigir siempre un wildcard. Mientras el sistema de tipos siga basado en herencia abierta, el catch-all es una regla simple y segura.

10.3. Protocolos y conformidad accidental

La conformidad estructural de protocolos reduce acoplamiento y elimina declaraciones explícitas de implementación. Sin embargo, también puede producir conformidad accidental: un tipo puede coincidir con un protocolo por tener métodos con nombres y firmas compatibles, aunque la intención conceptual sea distinta.

Este riesgo es inherente a los sistemas estructurales. El compilador verifica aridad, parámetros, retornos y reglas de varianza, pero no puede verificar intención semántica. Una posible extensión sería permitir conformidad explícita opcional, anotaciones de intención o herramientas de diagnóstico que adviertan coincidencias sospechosas.

11. Conclusiones

El compilador desarrollado presenta una solución completa y defendible para HULK desde la perspectiva de Compilación. El sistema implementa un pipeline por fases que transforma código fuente en LLVM IR y binario ejecutable, manteniendo representaciones intermedias

inspeccionables. La separación entre frontend, AST, semántica, HIR, IR, backend LLVM y runtime permite razonar sobre cada responsabilidad sin mezclar problemas de niveles distintos.

Las decisiones de diseño tomadas son coherentes con el lenguaje. El lexer generado mantiene una especificación léxica declarativa; la combinación de parser LL(1) y Pratt parser equilibra formalidad gramatical con expresiones ricas; el AST separa sintaxis concreta de estructura semántica; la HIR tipada fija decisiones de nombres y tipos antes del lowering; la IR propia actúa como puente entre HULK y LLVM; y el runtime C concentra operaciones de ejecución que serían costosas de emitir manualmente como LLVM textual.

Desde Lenguajes de Programación, la contribución principal es `match`. Esta extensión agrega clasificación declarativa de valores mediante patrones, bindings, narrowing de tipos y exhaustividad conservadora. Su diseño es suficientemente expresivo para mejorar programas orientados a objetos, pero suficientemente pequeño para integrarse mediante desugaring a construcciones existentes. La comparación con Rust, Swift, Kotlin, Scala y lenguajes funcionales muestra que `match` no pretende implementar pattern matching completo sobre ADTs, sino una variante ajustada al modelo de objetos de HULK.

El informe también delimita el alcance del sistema. El runtime no es una VM, la administración de memoria no es un GC general, la exhaustividad de `match` es conservadora, los protocolos pueden sufrir conformidad accidental, no existen genéricos definidos por el usuario y `define` se mantiene como mecanismo limitado. Estas limitaciones no invalidan el diseño; al contrario, muestran que el compilador prioriza claridad, corrección y completitud razonable dentro de un alcance académico.

En síntesis, el mayor valor técnico del proyecto está en la claridad del pipeline, en los contratos internos entre representaciones y en la integración de una extensión de lenguaje que afecta varias fases sin desestabilizar el backend. El resultado es un compilador modular, ejecutable mediante LLVM y preparado para seguir creciendo en semántica, runtime, optimización y diseño de lenguaje.

A. Comandos de uso recomendados

A.1. Construcción del compilador

```
cargo build --release -p hulk-driver
cp target/release/hulkc ./hulk
```

También puede usarse:

```
make build
```

A.2. Compilación de un programa HULK

```
./hulk examples/programa.hulk
./output
```

El driver genera temporalmente `output.ll` y lo pasa a `clang`. Además enlaza el runtime ubicado en `runtime/hulk_runtime.c` con `-lm` y produce `./output`.

A.3. Inspección por fases

```
./hulk programa.hulk --ast
./hulk programa.hulk --check
./hulk programa.hulk --hir
./hulk programa.hulk --ir
./hulk programa.hulk --emit-llvm
```

Estos comandos son útiles para demostrar que cada fase produce una representación observable y para localizar errores en el punto correcto del pipeline.

B. Programa demostrativo

Listing 13: Programa sugerido para demostrar mecanismos principales

```
function square(x: Number): Number => x * x;

type Shape() {
  area(): Number => 0;
}

type Circle(r: Number) inherits Shape {
  r: Number = r;
  area(): Number => 3 * square(self.r);
}

type Rectangle(w: Number, h: Number) inherits Shape {
  w: Number = w;
  h: Number = h;
  area(): Number => self.w * self.h;
}

function describe(s: Shape): String =>
  match (s) {
    Circle as c => "circle area = " @ c.area(),
    Rectangle as r => "rectangle area = " @ r.area(),
    _ => "unknown shape"
  };
```

```
let s: Shape = new Circle(4) in {  
    print(describe(s));  
    for (x in range(1, 4)) print(x);  
};
```

Este ejemplo permite defender varias partes del proyecto en una sola demostración: funciones globales, objetos, herencia, métodos, `self`, despacho dinámico, `match`, narrowing con `Circle` as `c`, concatenación de strings, llamadas a builtins, rangos e iteración mediante `for`.

C. Referencias consultadas

- [1] The Rust Project Developers. *The Rust Reference: Match expressions*. Disponible en: <https://doc.rust-lang.org/reference/expressions/match-expr.html>
- [2] Apple. *The Swift Programming Language: Patterns*. Disponible en: <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/patterns/>
- [3] JetBrains. *Kotlin Documentation: Control flow, when expressions*. Disponible en: <https://kotlinlang.org/docs/control-flow.html#when-expressions-and-statements>
- [4] Scala Documentation. *Pattern Matching*. Disponible en: <https://docs.scala-lang.org/tour/pattern-matching.html>
- [5] The Go Authors. *The Go Programming Language Specification: Interface types*. Disponible en: <https://go.dev/ref/spec>
- [6] Microsoft. *TypeScript Handbook: Type Compatibility*. Disponible en: <https://www.typescriptlang.org/docs/handbook/type-compatibility.html>
- [7] The Rust Project Developers. *The Rust Reference: Traits*. Disponible en: <https://doc.rust-lang.org/reference/items/traits.html>
- [8] Oracle. *The Java Language Specification, Chapter 9: Interfaces*. Disponible en: <https://docs.oracle.com/javase/specs/jls/se24/html/jls-9.html>